



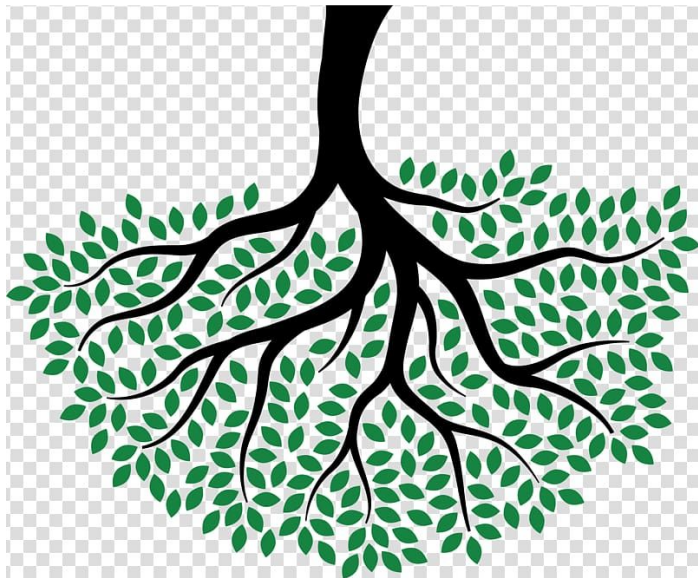
ED1 06: Árvores

Fabio Irigon Pereira

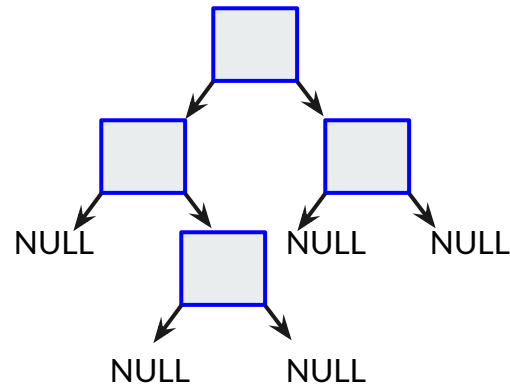
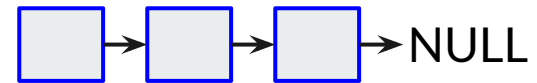
Árvores



Árvores



Lista



Conceito de Árvore

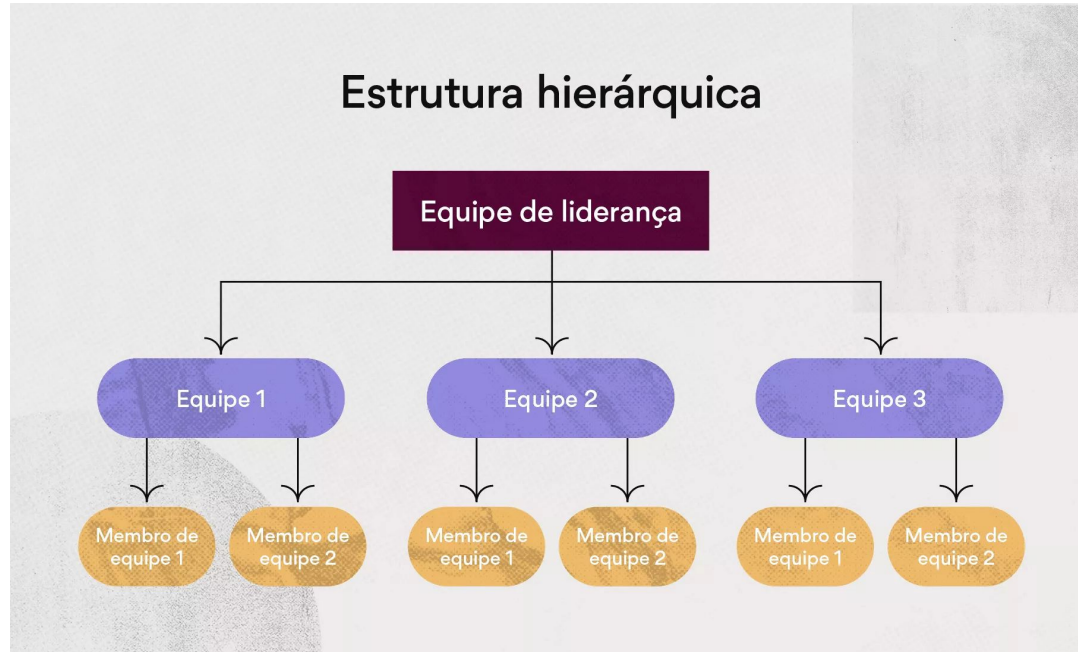


Uma **árvore** é uma estrutura hierárquica composta por nós. Cada nó pode ter zero ou mais filhos, formando uma estrutura não linear.

- **Raiz** (root): o primeiro nó da árvore.
- **Pai e filho** (parent/child): relação entre nós conectados.
- **Folha** (leaf): nó sem filhos.
- **Altura**: número de arestas no maior caminho da raiz até uma folha.
- **Grau do nó**: número de filhos diretos que o nó possui.

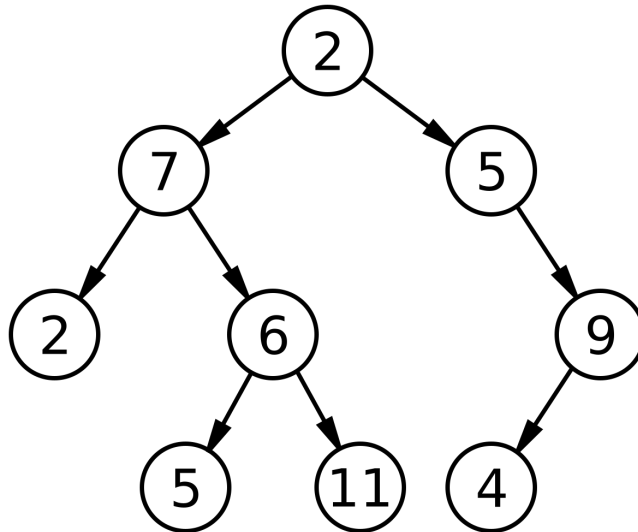
Exemplo de Árvore

Exemplo prático: Organização hierárquica de empresa.

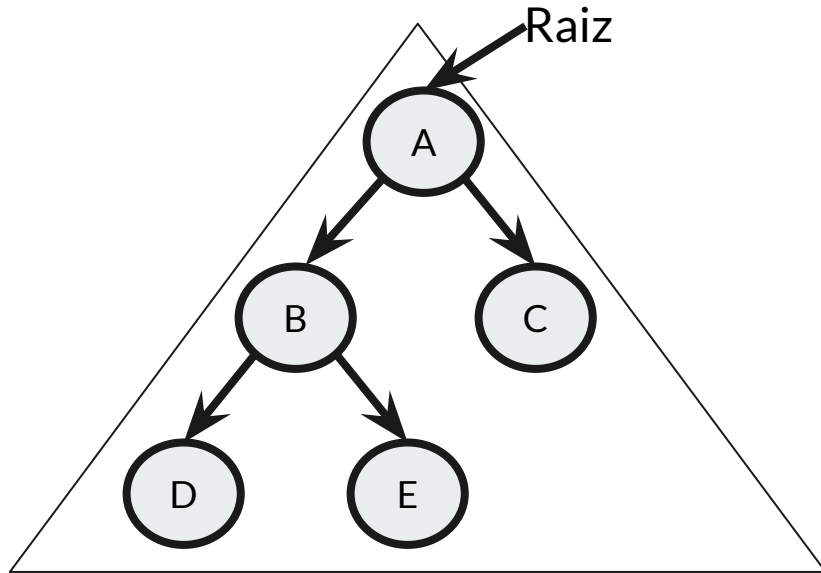


Árvores Binárias

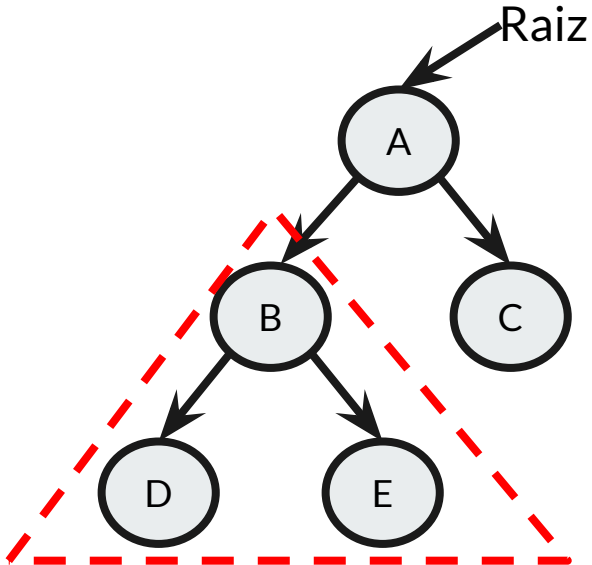
Uma árvore binária é um tipo especial de árvore onde cada nó pode ter no máximo dois filhos.



Travessia e recursão em árvores binárias



Travessia e recursão em árvores binárias



Ao percorrer uma árvore, cada filho pode ser visto como a raiz de uma sub-árvore.

Travessia e recursão em árvores binárias

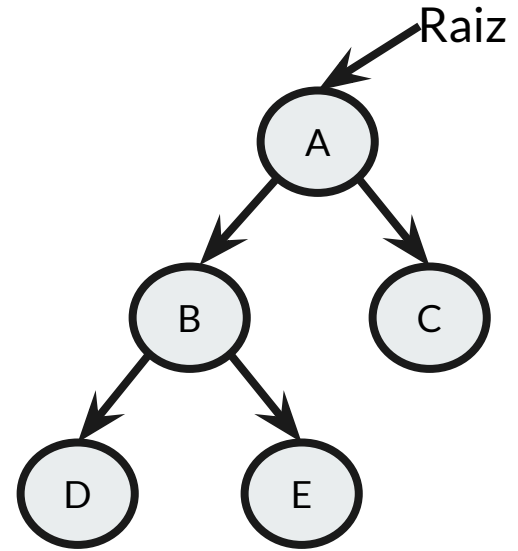


Usa recursividade para tratar cada filho como uma nova árvore.

- PréOrdem: Raiz, Esquerda, Direita.
- InOrdem: Esquerda, Raiz, Direita.
- PósOrdem: Esquerda, Direita, Raiz.

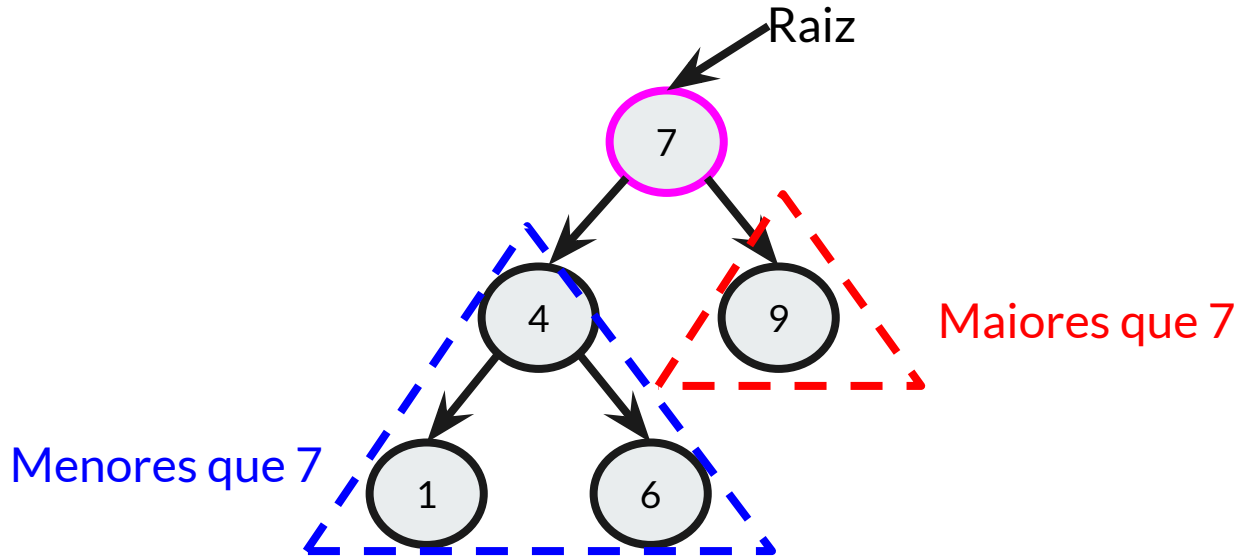
Travessia e recursão em árvores binárias

Qual seria a ordem de tratamento dos nós da árvore ao lado em cada um dos casos: PréOrdem, EmOrdem e PósOrdem?



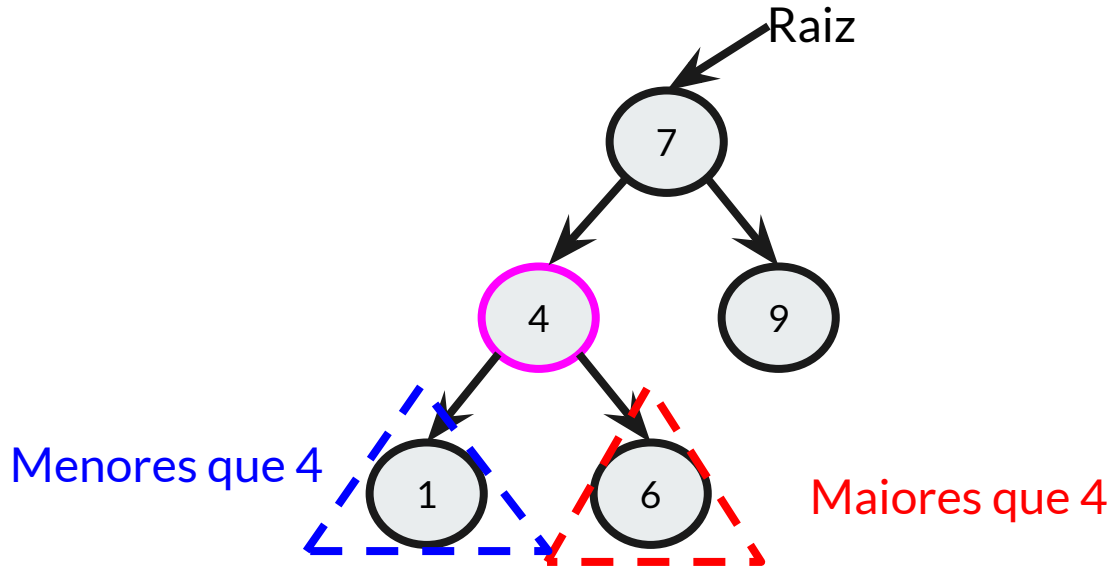
Árvores binária de busca

Árvore ordenada, onde os valores da sub-árvore a esquerda são menores do que o pai e os da direita são maiores.



Árvores binária de busca

Árvore ordenada, onde os valores da sub-árvore a esquerda são menores do que o pai e os da direita são maiores.



Operações em Árvore Binárias

fonte: <https://github.com/arbackes/Estrutura-de-Dados-em-C>

Definição do nó



```
struct NO{  
    int info;  
    struct NO *esq;  
    struct NO *dir;  
};
```

Impressão recursiva:



```
void preOrdem_ArvBin(ArvBin *raiz){
    if(raiz == NULL)
        return;
    if(*raiz != NULL){
        printf("%d\n", (*raiz)->info);
        preOrdem_ArvBin(&((*raiz)->esq));
        preOrdem_ArvBin(&((*raiz)->dir));
    }
}
```

Pré ordem: imprime o nó atual, imprime subárvore à esquerda, e imprime a subárvore à direita.

Impressão recursiva:



```
void preOrdem_ArvBin(ArvBin *raiz){
    if(raiz == NULL)
        return;
    if(*raiz != NULL){
        printf("%d\n", (*raiz)->info);
        preOrdem_ArvBin(&((*raiz)->esq));
        preOrdem_ArvBin(&((*raiz)->dir));
    }
}
```

```
void emOrdem_ArvBin(ArvBin *raiz){
    if(raiz == NULL)
        return;
    if(*raiz != NULL){
        emOrdem_ArvBin(&((*raiz)->esq));
        printf("%d\n", (*raiz)->info);
        emOrdem_ArvBin(&((*raiz)->dir));
    }
}
```

```
void posOrdem_ArvBin(ArvBin *raiz){
    if(raiz == NULL)
        return;
    if(*raiz != NULL){
        posOrdem_ArvBin(&((*raiz)->esq));
        posOrdem_ArvBin(&((*raiz)->dir));
        printf("%d\n", (*raiz)->info);
    }
}
```

Exercícios:



1. Somar todos os valores de uma árvore binária.
2. Contar o número de folhas de uma árvore binária.
3. Imprimir a altura de cada nó.
4. Retornar a altura de uma árvore.
5. Buscar um valor na árvore.

Árvores Binárias de busca (BST)

fonte: <https://github.com/arbackes/Estrutura-de-Dados-em-C>

Inserção em BST

```
int insere_ArvBin(ArvBin* raiz,
struct NO* novo){
    if(raiz == NULL)
        return 0;
    if(*raiz == NULL)
        *raiz = novo;
    else{
        ...
```

```
    else{
        struct NO* atual = *raiz;
        struct NO* ant = NULL;
        while(atual != NULL){
            ant = atual;
            if(valor == atual->info){
                return 0; //elemento já existe
            }

            if(valor > atual->info)
                atual = atual->dir;
            else
                atual = atual->esq;
        }
        if(valor > ant->info)
            ant->dir = novo;
        else
            ant->esq = novo;
    }
    return 1;
}
```

Remoção em BST



Casos:

- Nó sem filhos → apenas libera.
- Nó com um filho → substitui pelo filho.
- Nó com dois filhos → substitui pelo menor da sub-árvore direita (ou maior da esquerda).

Exercícios:



Usando o exemplo:

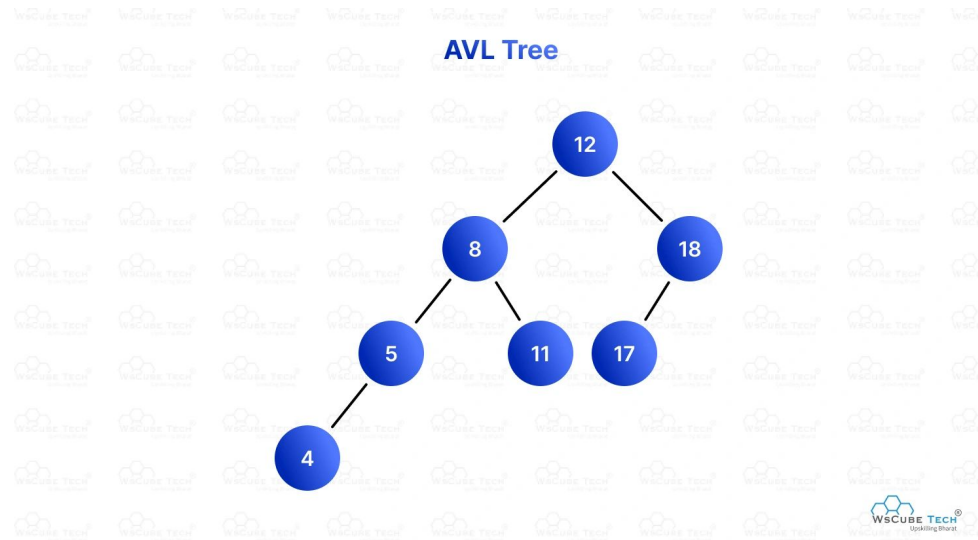
1. Encontrar o menor valor de uma árvore BST.
2. Contar o número de nós com apenas um filho.
3. Contar valores menores do que 20 evitando transitar em nós desnecessários).
4. Implementar rotina de remoção de nó pelo valor:
 - a. Encontrar nó a ser removido,
 - b. Encontrar o menor valor da sub-árvore direita,
 - c. Copiar o menor para o nó a ser removido
 - d. Remover o menor da direita.

Árvores balanceadas

fonte: <https://github.com/arbackes/Estrutura-de-Dados-em-C>

Árvores AVL

São árvores binárias balanceadas onde a diferença de altura entre a sub-árvore esquerda e direita de qualquer nó é no máximo 1.



Rebalanceamento



Rebalanceamento (rotações)

Casos e suas rotações:

- **LL (simples à direita)**
- **RR (simples à esquerda)**
- **LR (dupla: esquerda-direita)**
- **RL (dupla: direita-esquerda)**

Procedimento de inserção



Inserção com rebalanceamento

Após inserir normalmente, você:

1. Atualiza altura.
2. Verifica fator de balanceamento.
3. Aplica a rotação adequada se necessário.

Rotação simples

